Class 12: RNASeq Analysis

Laura Sun (PID:A17923552)

Table of contents

Background

Today we will analyze some RNASeq data from himes et al. on the effects of a common steroid (dexamethasone) on airway smooth muscle cells (ASM cells).

Our starting point is the "counts" data and "metadata" that contain the count values for each gene in their different experiments (i.e. cell lines with or without the drug).

Data import

Load in console: install.packages("BiocManager") BiocManager::install("DESeq2") n

```
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <- read.csv("airway_metadata.csv")
```

Let's take a look:

```
head(counts)
```

```
                SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR1039516
ENSG00000000003        723        486        904        445       1170
ENSG00000000005          0          0          0          0          0
ENSG00000000419        467        523        616        371        582
ENSG00000000457        347        258        364        237        318
ENSG00000000460         96         81         73         66        118
ENSG00000000938          0          0          1          0          2
                SRR1039517 SRR1039520 SRR1039521
```

```
ENSG00000000003       1097        806        604
ENSG00000000005          0          0          0
ENSG00000000419        781        417        509
ENSG00000000457        447        330        324
ENSG00000000460         94        102         74
ENSG00000000938          0          0          0
```

Q1: How many genes are in this dataset?

Number of genes:

```
nrow(counts)
```

```
[1] 38694
```

Experiments:

```
nrow(metadata)
```

```
[1] 8
```

Q2: How many 'control' cell lines do we have?

```
sum(metadata$dex=="control")
```

```
[1] 4
```

Toy differential gene expression

To start our analysis, let's calculate the mean counts for all genes in the "control" experiments.

1. Extract all control columns from the `counts` object.
2. Calculate the mean for all rows (i.e. genes) of these "control" columns.

3-4. Do the same for "treated" group 5. Compare these `control.mean` and `treated.mean` values.

```
control.inds <- metadata$dex=="control"
control.counts <- counts[,control.inds]
control.means <- rowSums(control.counts)/4 # Getting the average of each gene's 4 controls. (
```

Q3. How would you make the above code in either approach more robust? Is there a function that could help here?

We can use `rowMeans()` instead of `rowSums()`/counts to make it easier.

Q4. Follow the same procedure for the treated samples (i.e. calculate the mean per gene across drug treated samples and assign to a labeled vector called treated.mean)

```
treated.inds <- metadata$dex=="treated"
treated.counts <- counts[,treated.inds]
treated.means <- rowMeans(treated.counts)
```

Store these together for ease of bookkeeping as `meancounts`

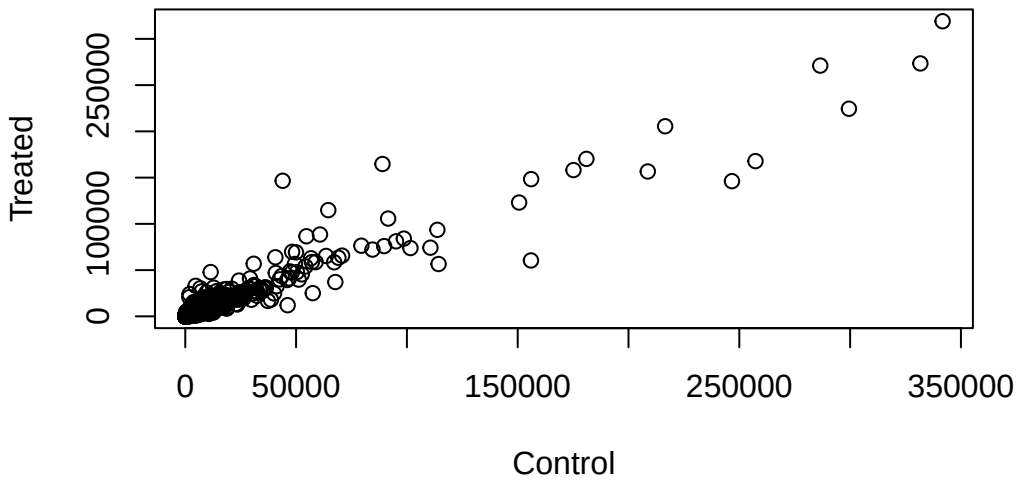
```
meancounts <- data.frame(control.means, treated.means)
head(meancounts)
```

```
                control.means treated.means
ENSG00000000003        900.75        658.00
ENSG00000000005          0.00          0.00
ENSG00000000419        520.50        546.00
ENSG00000000457        339.75        316.50
ENSG00000000460         97.25         78.75
ENSG00000000938          0.75          0.00
```

Make a plot of control vs. treated mean values for all genes.

Q5 (a). Create a scatter plot showing the mean of the treated samples against the mean of the control samples. Your plot should look something like the following.

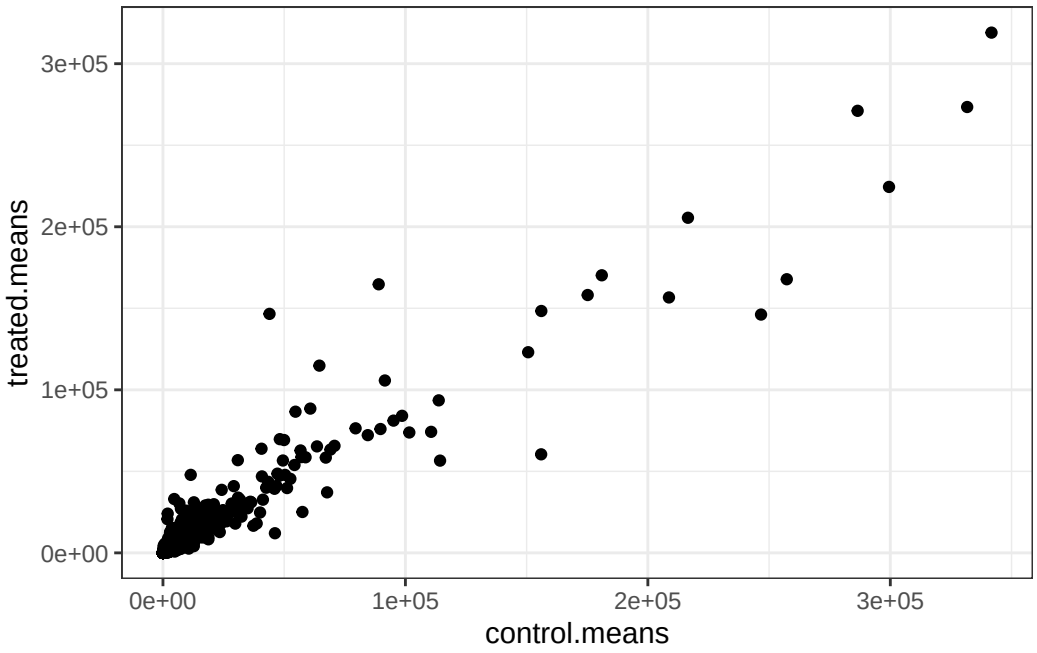
```
plot(meancounts[,1],meancounts[,2], xlab="Control", ylab="Treated")
```



Q5 (b).You could also use the ggplot2 package to make this figure producing the plot below. What geom_?() function would you use for this plot?

Use geom_point() for scatter plot.

```
library(ggplot2)
ggplot(meancounts, aes(x=control.means, y=treated.means)) + geom_point() + theme_bw()
```



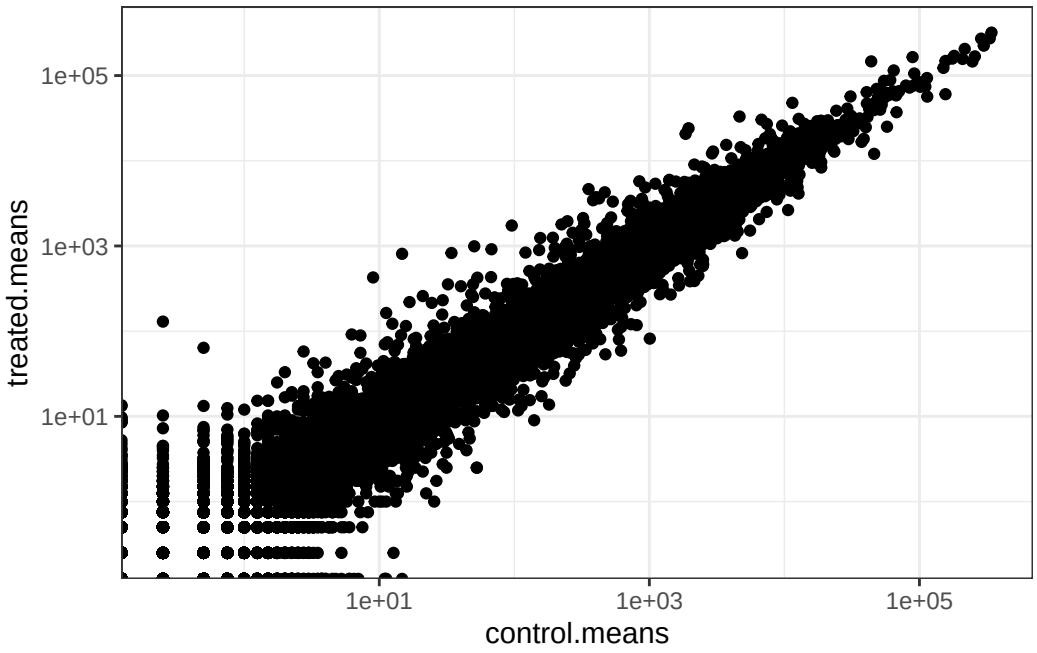
Q6. Try plotting both axes on a log scale. What is the argument to plot() that allows you to do this?

Use scale_x_log10() to adjust for skewed data.

```
ggplot(meancounts, aes(x=control.means, y=treated.means)) + geom_point() + theme_bw() + scale
```

```
Warning in scale_x_log10(): log-10 transformation introduced infinite values.
```

```
Warning in scale_y_log10(): log-10 transformation introduced infinite values.
```




```
# or plot(meancounts, log="xy")
```

We often talk about metrics like "log2 fold-change"

```
# control/treated
log2(10/10) # gives 0, so 0 change.
```

```
[1] 0
```

```
log2(40/10)
```

```
[1] 2
```

```
log2(10/40)
```

```
[1] -2
```

Let's calculate the log2 fold change for our treated oven control mean counts.

```
meancounts$log2fc <- log2(meancounts$treated.means/ meancounts$control.means)
# add a column.

head(meancounts)
```

```
                control.means treated.means      log2fc
ENSG00000000003        900.75        658.00 -0.45303916
ENSG00000000005          0.00          0.00         NaN
ENSG00000000419        520.50        546.00  0.06900279
ENSG00000000457        339.75        316.50 -0.10226805
ENSG00000000460         97.25         78.75 -0.30441833
ENSG00000000938          0.75          0.00        -Inf
```

```
zero.vals <- which(meancounts[,1:2]==0, arr.ind=TRUE)
to.rm <- unique(zero.vals[,1])
mycounts <- meancounts[-to.rm,]
head(mycounts)
```

```
                control.means treated.means      log2fc
ENSG00000000003        900.75        658.00 -0.45303916
ENSG00000000419        520.50        546.00  0.06900279
ENSG00000000457        339.75        316.50 -0.10226805
ENSG00000000460         97.25         78.75 -0.30441833
ENSG00000000971       5219.00       6687.50  0.35769358
ENSG00000001036       2327.00       1785.75 -0.38194109
```

> Q7: What is the purpose of the arr.ind argument in the which() function call above? Why would we then take the first column of the output and need to call the unique() function?

The arr.ind=TRUE with which() return the indices of TRUE values, showing the location of zeros. We ignore genes with zero counts. unique() helps to not count a row twice if it has zeros.

A common "rule of thumb" is a log2 full change cutoff of +2 and -2 to call genes "Up regulated" or "Down regulated".

> Q8: Number of "up regulated" genes:

```
sum(meancounts$log2fc > +2, na.rm=T)
```

```
[1] 1846
```

```
#na.rm=T means don't count the NAs.
```

Q9: Number of down regulated genes:

```
sum(meancounts$log2fc < -2, na.rm=T)
```

```
[1] 2212
```

Q10: We are missing statistical significance of the data. We don't know if the difference between control and treated are caused by random errors, skewed data, or there's a real significant difference. We can use something like two sample t-test to check for statistical significance.

DESeq2 analysis

```
library(DESeq2)
```

For DESeq analysis we need three things:

1. count values (`countData`)
2. metadata telling us about the columns in `countData` (`colData`)
3. design of the experiment (i.e. what do you want to compare)

Our first function from DESeq2 will setup the input required for analysis by storing all these 3 things together.

```
dds <- DESeqDataSetFromMatrix(countData = counts, colData = metadata, design = ~dex)
```

```
converting counts to integer mode
```

```
Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
design formula are characters, converting to factors
```

The main function in DESeq2 that runs the analysis is called `DESeq()`

```
dds <- DESeq(dds)
```

```
estimating size factors
```

```
estimating dispersions
```

```
gene-wise dispersion estimates
```

```
mean-dispersion relationship
```

```
final dispersion estimates
```

```
fitting model and testing
```

```
res <- results(dds)
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 6 rows and 6 columns
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195      -0.350703  0.168242 -2.084514 0.0371134
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160       0.206107  0.101042  2.039828 0.0413675
ENSG00000000457 322.664844       0.024527  0.145134  0.168996 0.8658000
ENSG00000000460  87.682625      -0.147143  0.256995 -0.572550 0.5669497
ENSG00000000938   0.319167      -1.732289  3.493601 -0.495846 0.6200029
                     padj
                <numeric>
ENSG00000000003  0.163017
ENSG00000000005        NA
ENSG00000000419  0.175937
ENSG00000000457  0.961682
ENSG00000000460  0.815805
ENSG00000000938        NA
```

```
36000*0.05
```

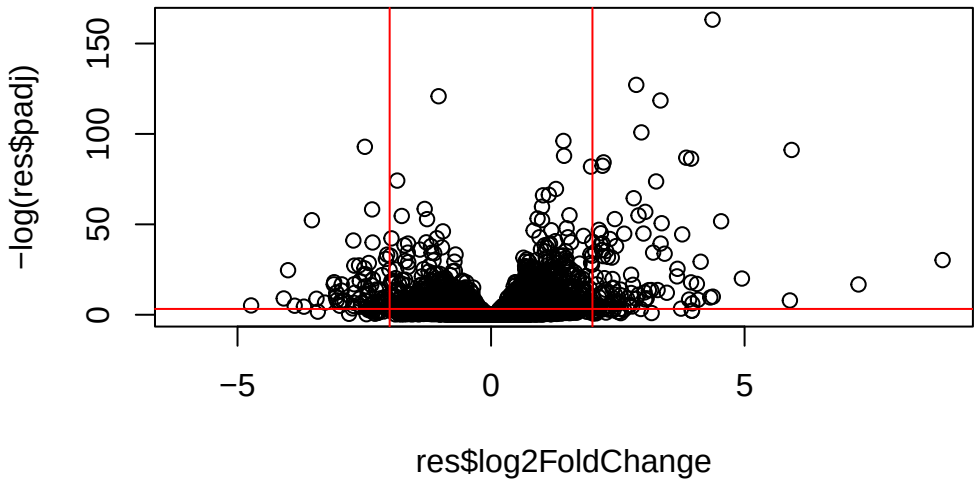
```
[1] 1800
```

Volcano Plot

This is a common summary result figure from these types of experiments and plot the log2 fold-change vs. the adjusted p-value.

```
plot(res$log2FoldChange, -log(res$padj))
# use -log to change the axis

abline(v=c(-2,2), col="red")
abline(h=-log(0.04), col="red")
```



Save our result

```
write.csv(res, file="my_results.csv")
```

Add gene annotation

To help make sense of our results and communicate them to other folk, we need to add more annotations to our main `res` object.

We will use two bioconductor packages to first map IDs to different formats including classic gene "symbol" gene name.

Install these with the following commands: `BiocManager::install("AnnotationDbi")` `BiocManager::install("org.Hs.eg.db")`

```
library("AnnotationDbi")
library("org.Hs.eg.db")
```

```
#mygenes$control <- maplds(org.Hs.eg.db, column="SYMBOL", #keys=row.names(mygenes), keytype"
```

Let's see what's in `org.Hs.eg.db` with the `columns()` function"

```
columns(org.Hs.eg.db)
```

```
 [1] "ACCNUM"       "ALIAS"        "ENSEMBL"      "ENSEMBLPROT"  "ENSEMBLTRANS"
 [6] "ENTREZID"     "ENZYME"       "EVIDENCE"     "EVIDENCEALL"  "GENENAME"
[11] "GENETYPE"     "GO"           "GOALL"        "IPI"          "MAP"
[16] "OMIM"         "ONTOLOGY"     "ONTOLOGYALL"  "PATH"         "PFAM"
[21] "PMID"         "PROSITE"      "REFSEQ"       "SYMBOL"       "UCSCKG"
[26] "UNIPROT"
```

We can translate or "map" IDs between any of these 26 databases using the `mapIDs()` function.

```
head(row.names(res))
```

```
[1] "ENSG00000000003" "ENSG00000000005" "ENSG00000000419" "ENSG00000000457"
[5] "ENSG00000000460" "ENSG00000000938"
```

```
res$symbol <- mapIds(keys=row.names(res), # our current IDs
       keytype="ENSEMBL", # the format of our IDs
       x=org.Hs.eg.db, # where to get the mappings from
       column="SYMBOL") # the format/DB to map to
```

```
'select()' returned 1:many mapping between keys and columns
```

```
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 6 rows and 7 columns
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195      -0.350703  0.168242 -2.084514 0.0371134
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160       0.206107  0.101042  2.039828 0.0413675
ENSG00000000457 322.664844       0.024527  0.145134  0.168996 0.8658000
ENSG00000000460  87.682625      -0.147143  0.256995 -0.572550 0.5669497
ENSG00000000938   0.319167      -1.732289  3.493601 -0.495846 0.6200029
                     padj      symbol
                <numeric> <character>
ENSG00000000003  0.163017      TSPAN6
ENSG00000000005        NA        TNMD
ENSG00000000419  0.175937        DPM1
ENSG00000000457  0.961682       SCYL3
ENSG00000000460  0.815805       FIRRM
ENSG00000000938        NA         FGR
```

Add the mappings for "GENENAME" and "ENTREZID" and store as `res$genename` and `res$entrez`

```
res$genename <- mapIds(keys=row.names(res), # our current IDs
       keytype="ENSEMBL", # the format of our IDs
       x=org.Hs.eg.db, # where to get the mappings from
       column="GENENAME") # the format/DB to map to
```

```
'select()' returned 1:many mapping between keys and columns
```

```
res$entrez <- mapIds(keys=row.names(res), # our current IDs
       keytype="ENSEMBL", # the format of our IDs
       x=org.Hs.eg.db, # where to get the mappings from
       column="ENTREZID") # the format/DB to map to
```

```
'select()' returned 1:many mapping between keys and columns
```

Pathway Analysis

There are lots of bioconductor packages to do this type of analysis. For now let's try one called **gage**. Again, we need t o install this if we don't have it already.

BiocManager::install("gage") BiocManager::install("gageData") BiocManager::install("pathview")

```
library(gage)
library(gageData)
library(pathview)
```

To use **gage** I need two things

- a named vector of fold-change DEGs (our geneset of interest)
- a set of pathways of genesets to use for

```
x <- c("barry"=5, "lisa"=10)
x
```

```
barry  lisa
    5    10
```

```
names(x) <- c("low", "high")
x
```

```
 low high
   5   10
```

```
foldchanges <- res$log2FoldChange
names(foldchanges) <- res$entrez
head(foldchanges)
```

```
       7105       64102        8813       57147       55732        2268
-0.35070296          NA  0.20610728  0.02452701 -0.14714263 -1.73228897
```

```
data(kegg.sets.hs)
keggres=gage(foldchanges,gsets=kegg.sets.hs)
```

In our results object we have: